

Softwarequalität

HOR-TInf2020/1

Gruppenarbeit

Hyperformer_ContactSplitter

Anwendungsdokumentation

12.05.2023

Dipl.-Math. Christian Hübscher

Inhaltsverzeichnis

Hyperformer_ContactSplitter	3
Allgemeine Prinzipien	3
Composite Components und Dependency Inversion	3
Dependency Injection	3
Schichtenarchitektur	3
MVVM	3
CleanCode	3
Anmerkungen zum Parser	4
Konkreter Projektaufbau	4

Abbildungsverzeichnis

Abbildung 1: Klassendiagramm Teil eins	6
Abbildung 2: Klassendiagramm Teil zwei	7
Abbildung 3: Klassendiagramm Teil drei	8

Hyperformer_ContactSplitter

Dieses Dokument soll Aufschluss über Architekturentscheidungen geben.

Allgemeine Prinzipien

Im Folgenden allgemeine Architektur-Prinzipien.

Composite Components und Dependency Inversion

Die Composite Components Architektur zielt auf eine Entkopplung von Abhängigkeiten auf Komponentenebene ab. Eine Komponente besteht in .NET immer aus zwei Assemblies: Ein Assembly enthält die Implementierungen, das andere die Schnittstellen und Datenklassen (Contract).

Assemblies besitzen dabei immer nur Abhängigkeiten zu einer Contract Assembly (Dependency Inversion). Dies ermöglicht eine Austauschbarkeit auf Komponentenebene. Dieses Design wird vom Softwarearchitekt David Tilke (<https://www.david-tielke.de>) für .NET-Businessanwendungen empfohlen und orientiert sich an dem ISO-9126 Qualitätsmodell.

Dependency Injection

Das Mapping von Contracts und Implementierungen erfolgt über einen Dependency Injection Container, dessen Konfiguration sich im Hauptprojekt (ContactSplitter) befindet. Das Hauptprojekt konsumiert alle Komponenten und führt entsprechende Kompositionen durch. Diese Schritte erhöhen die Testbarkeit, Wiederverwendbarkeit und Austauschbarkeit der Komponenten und Klassen und erhöhen jeweils die Modularität.

Schichtenarchitektur

Die Schichtenarchitektur sieht vor, dass eine Schicht jeweils nur eine Abhängigkeit zur unmittelbar darunterliegenden Schicht besitzt. Im Projekt gibt es die Schichten UI, Logik und Repository, welche sich in den Assemblies: ContactSplitter.Control (UI), ContactParser.Contracts und ContractParser (Logik) und ContactSplitter.Datastorage (Repository) manifestieren.

MVVM

Die Model-View-ViewModel bezieht sich auf das Zusammenspiel von UI, Daten und UI-Logik. Durch Bindung an ein ViewModel findet die Entkopplung von UI-Design und UI-Logik statt. Somit kann das ViewModel leicht ausgetauscht werden.

CleanCode

Zur Einhaltung von CleanCode und entsprechender Codierungsrichtlinien, wurde neben sorgfalt der Entwickler, das Werkzeug Resharper verwendet. Dieses Werkzeug führt statische Codeanalysen durch

und kann mittels syntaktischer semantischer Analyse Antipatterns, Inkonsistenzen, mögliche Vereinfachungen sowie Verletzungen von Codierungsrichtlinien (Namenskonventionen, Einrückung, Formatierung...) erkennen. Die verwendeten Codierungsrichtlinien sind die von der Firma JetBrains empfohlenen und im Resharper vorkonfigurierten Richtlinien: <https://www.jetbrains.com/dotnet/guide/tutorials/resharper-essentials/>

All diese Prinzipien sorgen für bessere Wiederverwendbarkeit, Austauschbarkeit, Testbarkeit, Erweiterbarkeit und Modularität (getrennte Entwicklung).

Anmerkungen zum Parser

Für eine Verbesserte Nutzererfahrung gibt es zwei Varianten des Parsers, welche der Nutzer in den Einstellungen auswählen kann.

Offline (Default) Parser: Parst die Nutzereingabe mittels logischer programmatischer Textanalyse. Liefert korrekte Ergebnisse, kann jedoch keine Inferenz mit Weltwissen betreiben (z.B. von dem Namen ‚Hans‘ auf das Geschlecht ‚männlich‘ schließen).

Online (GPT) Parser: Verwendet das Sprachmodell GPT3.5. Durch das breite Weltwissen, welches das vortrainierte Natural Language Processing Modell besitzt, kann neben dem Parsen auch logische Inferenz betrieben werden. So kann beispielsweise von dem Namen ‚Hans‘ auf das Geschlecht ‚männlich‘ geschlossen werden. Dieser Parser benötigt eine Internetverbindung und ist nicht streng deterministisch. Die Ergebnisse können bei gleichen Eingaben unterschiedlich sein.

Konkreter Projektaufbau

Projektmappe Hyperformer_ContactSplitter

__ ContactsParser.Contracts	(Siehe Abschnitt "Composite Components")
__ ContactParser	(Logik des klassischen Parsens)
__ GPTContactParser	(Logik des ChatGPT-API Parsers)
__ ContactSplitter.Control	(Wiederverwendbares UI-Element, welches Darstellung, Eingabe, Ausgabe regelt)
__ ContactSplitter	(Hauptprojekt)
__ ContactSplitter.DataStorage	(Datenhaltung aller Kontakte usw.)
__ ContactSplitter.Tests	(Testprojekt zur Validierung)

Es handelt sich um jeweils einzelne Projekte (Assemblies), welche in einer Projektmappe (Solution) zusammengefasst sind und interagieren. Auf den folgenden Seiten sind Abbildungen zu sehen, welche die einzelnen Klassendiagramme der Projekte darstellen.

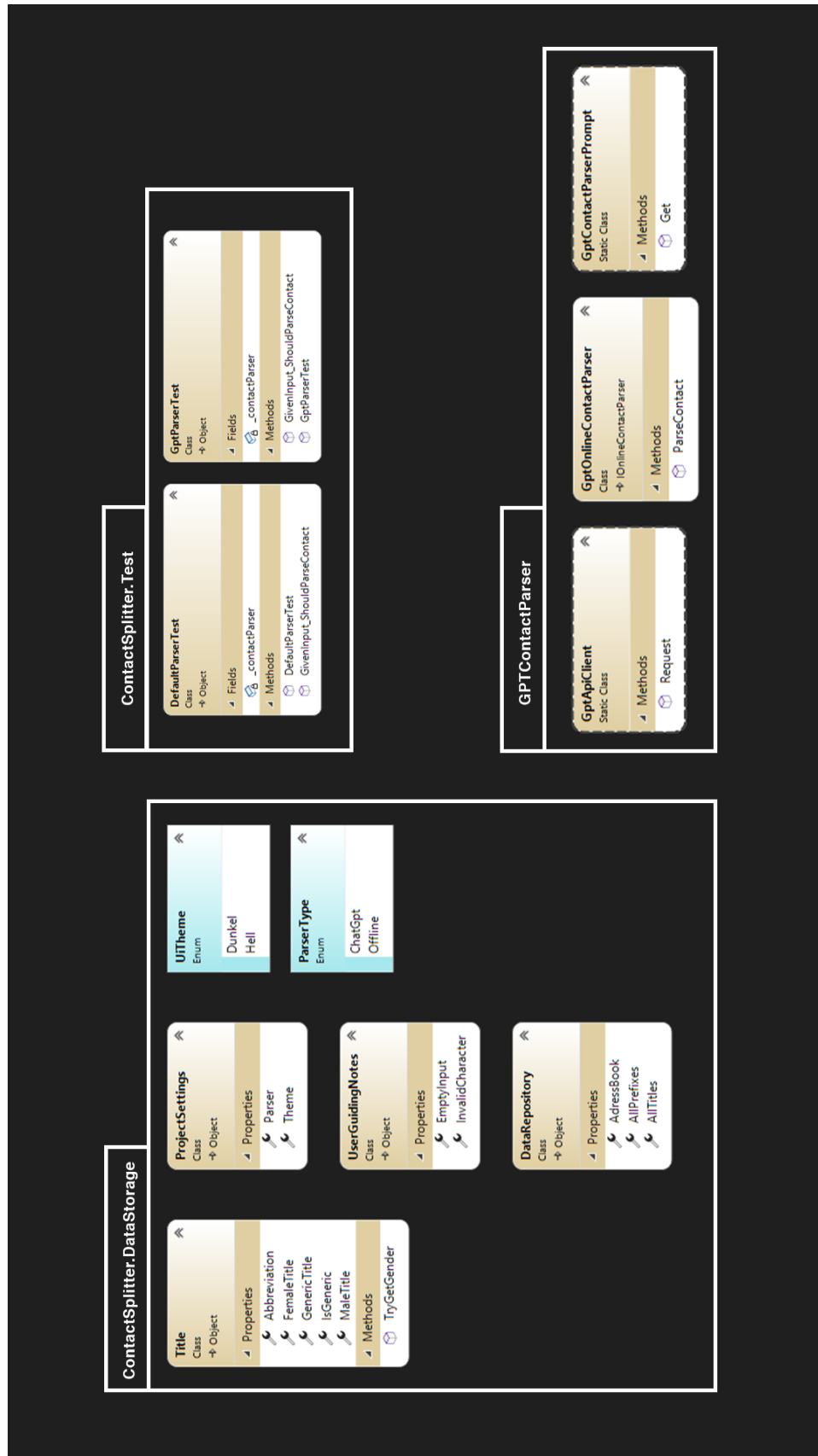


Abbildung 1: Klassendiagramm Teil eins

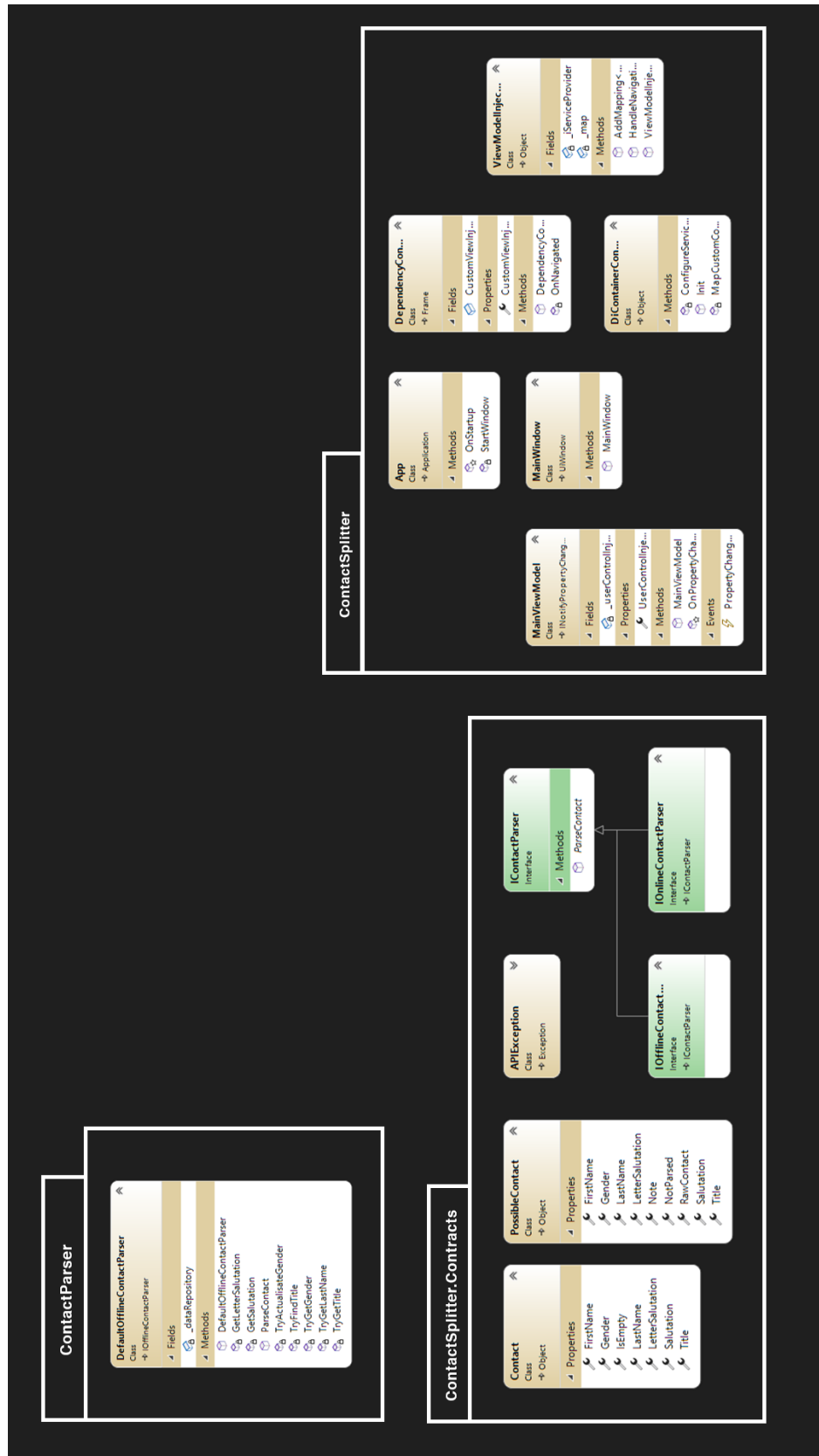


Abbildung 2: Klassendiagramm Teil zwei

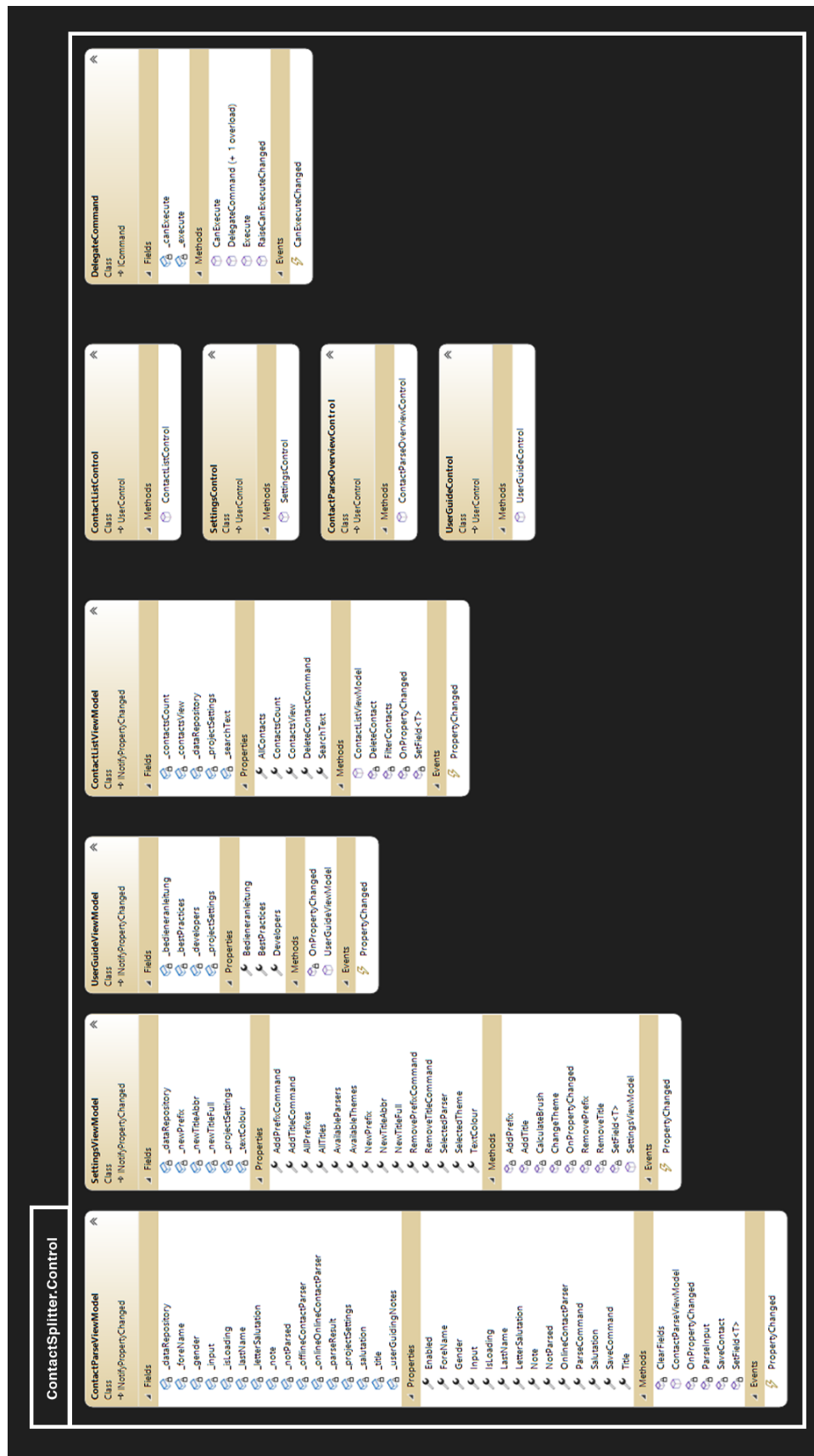


Abbildung 3: Klassendiagramm Teil drei